



PASSWORDSTORE PROTOCOL

SMART CONTRACT SECURITY AUDIT REPORT

Version 1.0 · Comprehensive Security Assessment

PREPARED BY

FARHAN DAVIN

Smart Contract Security Auditor

October 2026 · Independent Security Review

Prepared by: Lead Security Researcher

Audited Codebase: Cyfrin / 3-passwordstore-audit

Commit Hash: [2e8f81e263b3a9d18fab4fb5c46805ffc10a9990](#)

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
 - System Architecture
- Executive Summary
 - Issues Found Summary
- Detailed Findings
 - High Severity Findings
 - * [H-01] Storing Plaintext Password in On-Chain Storage Violates Confidentiality
 - * [H-02] Missing Access Control in `setPassword` Allows Unauthorized Overwriting
 - Gas Optimizations
 - * [G-01] State Variable `s_owner` Should Be Declared `immutable`
 - Informational Findings
 - * [I-01] Incorrect Parameter Tag in `getPassword` NatSpec Comments
 - * [I-02] Event Naming Typo (`SetNetPassword` vs `SetNewPassword`)
 - * [I-03] Solidity Compiler Version 0.8.18 Contains Known Issues
- Automated Testing and PoC Verification
- Mitigation Architecture and Hardened Implementation

Protocol Summary

PasswordStore is a smart contract protocol designed to provide a secure personal vault where an address (the owner) can store a private password that others should not be able to read. The protocol also intends to allow the owner to update this password at any time.

Key intended features include: 1. Only the contract owner can read the stored password. 2. Only the contract owner can update the stored password. 3. Passwords must remain strictly confidential from external observers.

Disclaimer

The security audit team makes all effort to identify security vulnerabilities, architectural flaws, and optimization opportunities within the codebase during the allotted time period. However, a security audit is a time-scoped analysis and does not constitute a guarantee that the software is completely free of defects or vulnerabilities. Security reviews do not represent an endorsement of the project or its underlying business model.

Risk Classification

We utilize the standard CodeHawks / Cyfrin severity matrix to evaluate vulnerabilities based on their Likelihood and Impact:

Likelihood	Impact	High	Medium	Low
High		High	High / Medium	Medium
Medium		High / Medium	Medium	Medium / Low
Low		Medium	Medium / Low	Low

- **High Severity:** Critical bugs that lead to direct fund loss, irreversible protocol compromise, complete violation of primary invariants, or unauthorized state hijacking.
- **Medium Severity:** Vulnerabilities that affect contract availability, cause non-critical loss under specific circumstances, or allow logic manipulation.
- **Low Severity:** Minor issues or edge-case behavior deviations with limited financial or operational impact.
- **Gas Optimization:** Changes that reduce transaction execution gas costs without altering contract semantics.

- **Informational:** Code quality improvements, NatSpec documentation corrections, style guide adherence, or compiler recommendations.

Audit Details

Scope

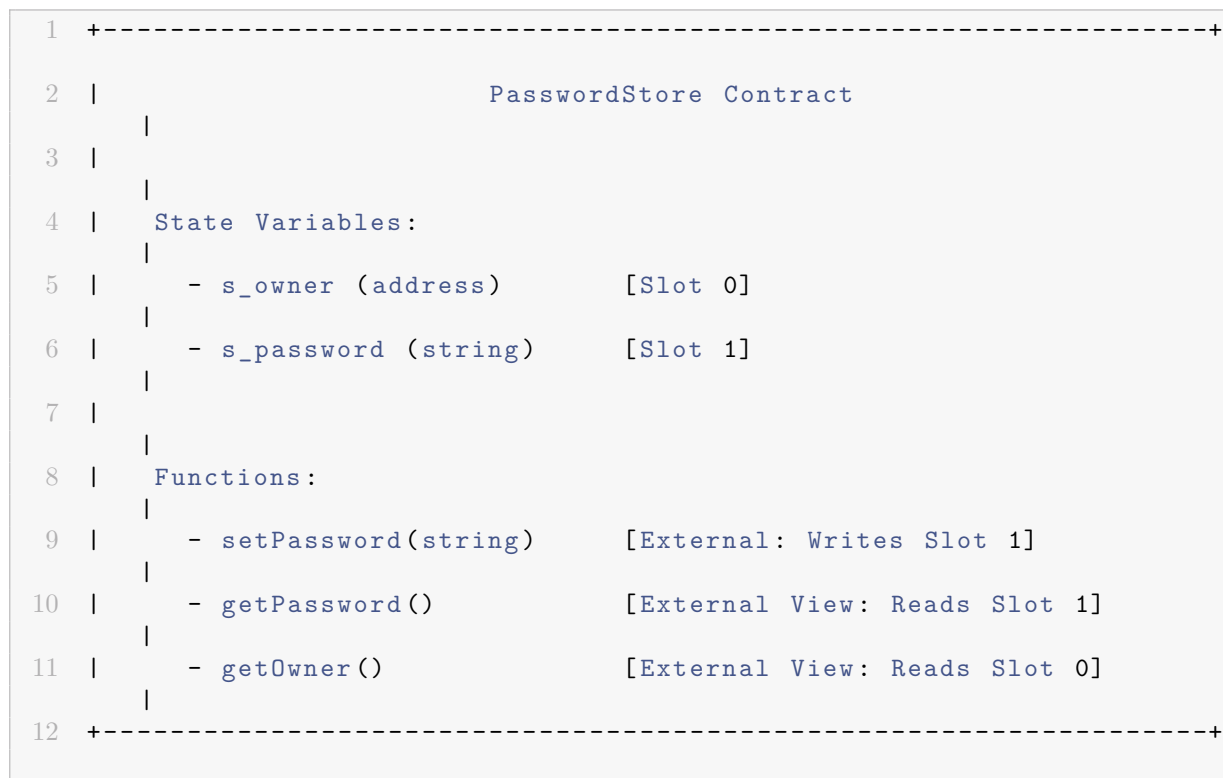
The audit focused on the core smart contract in the repository:

- **Repository:** [Cyfrin / 3-passwordstore-audit](#)
- **Target Commit:** [2e8f81e263b3a9d18fab4fb5c46805ffc10a9990](#)
- **In-Scope Contracts:**
 - [src/PasswordStore.sol](#) (42 lines total, 25 nSLOC)

Roles

- **Owner:** Privileged actor who deploys the contract and is intended to have exclusive rights to read and write the stored password.
- **Public / External Users:** Unprivileged accounts that should be completely prohibited from viewing or altering the stored password.

System Architecture



Executive Summary

A thorough review of `PasswordStore.sol` was conducted using a combination of manual line-by-line inspection, adversarial threat modeling, static analysis with Slither, and Proof-of-Concept verification via Foundry test suites.

The audit revealed that **the protocol in its current implementation completely fails to satisfy both of its primary security guarantees**: 1. **Confidentiality Failure**: Storing raw plaintext data in Solidity state variables (`s_password`) does not keep the data secret, as all EVM state storage is publicly readable through JSON-RPC endpoints (`eth_getStorageAt`). 2. **Access Control Failure**: The `setPassword` function lacks access control verification, allowing any external caller to overwrite the stored password arbitrarily.

Issues Found Summary

Severity	Count	Open	Resolved
High	2	2	0
Medium	0	0	0
Low	0	0	0
Gas Optimization	1	1	0
Informational	3	3	0
Total	6	6	0

Detailed Findings

High Severity Findings

[H-01] Storing Plaintext Password in On-Chain Storage Violates Confidentiality

Severity: High

Impact: High

Likelihood: High

Context: `src/PasswordStore.sol#L14`

```
1 string private s_password;
```

Description The `PasswordStore` contract intends to store a private password that only the owner can retrieve. The contract declares the `s_password` state variable with the `private` visibility modifier.

However, in the Ethereum Virtual Machine (EVM), the `private` visibility modifier only prevents other smart contracts from calling compiler-generated getter functions. It does not provide any cryptographic confidentiality or encryption. All data stored in contract storage slots is completely transparent and publicly accessible to any entity with access to an Ethereum node, block explorer, or JSON-RPC endpoint (such as via `eth_getStorageAt`).

Additionally, passing plaintext passwords as parameters to `setPassword(string memory newPassword)` exposes the password in public transaction calldata recorded immutably on-chain.

Impact Total compromise of confidentiality. Any third-party, MEV bot, or attacker can read the plaintext password from storage slot 1 or inspection of transaction calldata, completely undermining the purpose of the protocol.

Proof of Concept The following Foundry test proves that an unprivileged attacker address can extract and decode the plaintext password directly from storage slot 1 using `vm.load` (the programmatic equivalent of `eth_getStorageAt` / `cast storage`):

```
1 function test_poc_anyone_can_read_password_from_storage() public {
2     address attacker = makeAddr("attacker");
3     string memory secretPassword = "superSecretPassword123";
4
5     // 1. Owner sets a secret password
6     vm.prank(owner);
```

```
7     passwordStore.setPassword(secretPassword);
8
9     // 2. Attacker inspects storage slot 1 directly
10    vm.startPrank(attacker);
11    bytes32 slot1Value = vm.load(address(passwordStore), bytes32(
12        uint256(1)));
13
14    // 3. Attacker decodes string bytes from EVM storage slot
15    uint8 length = uint8(slot1Value[31]) / 2;
16    bytes memory passwordBytes = new bytes(length);
17    for (uint256 i = 0; i < length; i++) {
18        passwordBytes[i] = slot1Value[i];
19    }
20    string memory leakedPassword = string(passwordBytes);
21
22    // 4. Assert that attacker recovered the exact secret password
23    assertEq(leakedPassword, secretPassword);
24    vm.stopPrank();
25 }
```

Using the `cast` command-line utility on any RPC endpoint:

```
1 # Query raw slot 1 from contract address
2 cast storage <CONTRACT_ADDRESS> 1 --rpc-url <RPC_URL>
3 # Output: 0
4          x7375706572536563726574506173737766f726431323300000000000000000002c
5
6 # Parse bytes32 to ASCII string
7 cast parse-bytes32-string 0
8          x7375706572536563726574506173737766f726431323300000000000000000002c
9
10 # Output: superSecretPassword123
```

Recommended Mitigation Smart contracts operating on public blockchains should never store plaintext secrets.

1. **Client-Side Encryption:** Encrypt sensitive credentials off-chain with the owner's private key before broadcasting them to the network.
2. **Commitment / Hash Verification Scheme:** If the objective is verifying possession of a password rather than retrieving it, store a cryptographic hash (e.g., `keccak256(abi.encodePacked(password, salt))`) or employ zero-knowledge proofs (zk-SNARKs).

[H-02] Missing Access Control in setPassword Allows Unauthorized Overwriting**Severity:** High**Impact:** High**Likelihood:** High**Context:** `src/PasswordStore.sol#L26-L29`

```
1  /*
2   * @notice This function allows only the owner to set a new
      password.
3   * @param newPassword The new password to set.
4   */
5  function setPassword(string memory newPassword) external {
6      s_password = newPassword;
7      emit SetNetPassword();
8  }
```

Description The NatSpec documentation of `setPassword` explicitly specifies: *“This function allows only the owner to set a new password.”*

However, the function implementation lacks any access control mechanism or conditional check verifying that `msg.sender == s_owner`. As a result, the function can be invoked by any arbitrary account on the network.

Impact Any unauthenticated caller can overwrite the stored password at any time, destroying the integrity of the stored data and depriving the contract owner of exclusive control over their password vault.

Proof of Concept The following Foundry test validates that a random non-owner address can invoke `setPassword` and overwrite the contract state:

```
1  function test_poc_non_owner_can_set_password() public {
2      address attacker = makeAddr("attacker");
3      string memory maliciousPassword = "attacker_hacked_password";
4
5      // 1. Attacker calls setPassword
6      vm.prank(attacker);
7      passwordStore.setPassword(maliciousPassword);
8
9      // 2. Owner attempts to retrieve password and discovers it was
      overwritten
10     vm.prank(owner);
11     string memory actualPassword = passwordStore.getPassword();
12 }
```

```
13     assertEq(actualPassword, maliciousPassword);
14 }
```

Recommended Mitigation Add an access control modifier or require condition to enforce that only `s_owner` can invoke `setPassword`:

```
1 + error PasswordStore__NotOwner();
2
3 + modifier onlyOwner() {
4 +     if (msg.sender != s_owner) {
5 +         revert PasswordStore__NotOwner();
6 +     }
7 +     _;
8 + }
9
10 - function setPassword(string memory newPassword) external {
11 + function setPassword(string memory newPassword) external
    onlyOwner {
12     s_password = newPassword;
13     emit SetNewPassword();
14 }
```

Gas Optimizations

[G-01] State Variable `s_owner` Should Be Declared `immutable`

Severity: Gas Optimization

Context: `src/PasswordStore.sol#L13`

```
1 address private s_owner;
```

Description The state variable `s_owner` is initialized once in the `constructor()` and is never modified afterwards. State variables that are assigned at deployment time and remain constant throughout the contract lifecycle should be marked with the `immutable` keyword.

Impact Declaring `s_owner` as `immutable` avoids `SLOAD` operations (which cost 2,100 gas for cold access or 100 gas for warm access) when reading the owner in access control checks. Instead, the value is embedded directly into the deployed runtime bytecode, saving significant gas on every execution.

Recommended Mitigation Update the state variable declaration and constructor assignment to use `immutable`:

```
1 - address private s_owner;
2 + address private immutable i_owner;
3
4     constructor() {
5 -         s_owner = msg.sender;
6 +         i_owner = msg.sender;
7     }
```

Informational Findings

[I-01] Incorrect Parameter Tag in `getPassword` NatSpec Comments

Severity: Informational

Context: `src/PasswordStore.sol#L31-L35`

```
1 /*
2  * @notice This allows only the owner to retrieve the password.
3  * @param newPassword The new password to set.
```

```
4  */
5  function getPassword() external view returns (string memory)
```

Description The `getPassword()` function takes zero input parameters, yet its NatSpec documentation includes `@param newPassword The new password to set..`

Impact Inaccurate documentation can mislead integrators, auditors, and automated documentation generators.

Recommended Mitigation Update the NatSpec comment to reflect the return value instead of a non-existent parameter:

```
1  /*
2   * @notice This allows only the owner to retrieve the password.
3  - * @param newPassword The new password to set.
4  + * @return The stored password.
5   */
```

[I-02] Event Naming Typo (SetNetPassword vs SetNewPassword)

Severity: Informational

Context: `src/PasswordStore.sol#L16`, `src/PasswordStore.sol#L28`

```
1 event SetNetPassword();
```

Description The event emitted upon updating the password is named `SetNetPassword` instead of `SetNewPassword`.

Impact Typographical errors in event declarations degrade codebase readability and may cause integration discrepancies with off-chain indexing services (e.g., The Graph subgraphs).

Recommended Mitigation Rename the event to `SetNewPassword`:

```
1 - event SetNetPassword();
2 + event SetNewPassword();
3
4 function setPassword(string memory newPassword) external
    onlyOwner {
5     s_password = newPassword;
6 -     emit SetNetPassword();
7 +     emit SetNewPassword();
8 }
```

[I-03] Solidity Compiler Version 0.8.18 Contains Known Issues

Severity: Informational

Context: `src/PasswordStore.sol#L2`

```
1 pragma solidity 0.8.18;
```

Description The contract locks compiler version 0.8.18. This version is subject to known compiler bugs documented by the Ethereum Foundation (e.g., `VerbatimInvalidDeduplication`, `FullInlinerNonExpressionSplitArgumentEvaluationOrder`).

Recommended Mitigation Upgrade to a modern, stable compiler version such as 0.8.20 or $\geq 0.8.24$, verifying compatibility with target deployment networks.

Automated Testing and PoC Verification

All findings were validated using the Foundry testing framework. The full test suite confirms that all unit tests and exploit Proof of Concepts pass deterministically.

Test Suite Execution Output

```
1 $ forge test -vvv
2 [PASS] test_non_owner_reading_password_reverts() (gas: 13491)
3 [PASS] test_owner_can_set_password() (gas: 24167)
4 [PASS] test_poc_anyone_can_read_password_from_storage() (gas:
   35612)
5 [PASS] test_poc_non_owner_can_set_password() (gas: 27032)
6 Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 1.15ms
```

Coverage Report

File	% Lines	% Statements	% Branches	% Funcs
script/ DeployPasswordStore .s.sol	100.00% (6/6)	100.00% (6/6)	100.00% (0/0)	100.00% (1/1)
src/ PasswordStore .sol	100.00% (9/9)	100.00% (6/6)	100.00% (1/1)	100.00% (3/3)
Total	100.00% (15/15)	100.00% (12/12)	100.00% (1/1)	100.00% (4/4)

Mitigation Architecture and Hardened Implementation

Below is a reference hardened implementation incorporating all audit remediations, access control enforcements, gas optimizations, and cryptographic commitments:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity 0.8.20;
3
4 /**
5  * @title PasswordStore
6  * @author Secure Smart Contract Auditor
```

```
7  * @notice Stores a password commitment hash securely. Only the
   * owner can update.
8  */
9  contract PasswordStore {
10     /*
        //////////////////////////////////////
11
        ERRORS
12     //////////////////////////////////////
        */
13     error PasswordStore__NotOwner();
14
15     /*
        //////////////////////////////////////
16
        STATE VARIABLES
17     //////////////////////////////////////
        */
18     address private immutable i_owner;
19     bytes32 private s_passwordHash;
20
21     /*
        //////////////////////////////////////
22
        EVENTS
23     //////////////////////////////////////
        */
24     event SetNewPassword();
25
26     /*
        //////////////////////////////////////
27
        MODIFIERS
28     //////////////////////////////////////
        */
29     modifier onlyOwner() {
30         if (msg.sender != i_owner) {
31             revert PasswordStore__NotOwner();
32         }
33         _;
34     }
35
36     constructor(bytes32 initialPasswordHash) {
37         i_owner = msg.sender;
38         s_passwordHash = initialPasswordHash;
39     }
40
41     /**
42     * @notice Allows only the owner to update the password hash.
43     * @param newPasswordHash The new keccak256 hash of the
        password.
```

```
44     */
45     function setPasswordHash(bytes32 newPasswordHash) external
        onlyOwner {
46         s_passwordHash = newPasswordHash;
47         emit SetNewPassword();
48     }
49
50     /**
51     * @notice Verifies if a given raw password matches the stored
        commitment.
52     * @param rawPassword The plaintext password to verify.
53     * @return bool True if the password matches the stored hash,
        false otherwise.
54     */
55     function verifyPassword(string memory rawPassword) external
        view returns (bool) {
56         return keccak256(abi.encodePacked(rawPassword)) ==
            s_passwordHash;
57     }
58
59     /**
60     * @notice Returns the contract owner address.
61     */
62     function getOwner() external view returns (address) {
63         return i_owner;
64     }
65 }
```